

AI GENERALIST ACCELERATOR · 14-DAY AI SPRINT

Day Eleven

The Claude Ecosystem: Chat, Design, Code, and Cowork, and the Hackathon Opens

A cleaned, detailed record of the live session and the hackathon briefing

Topics covered: Four Friends · Requirements From Claude Chat · A Prototype in Claude Design · Cowork and the Messy Folder · Building in Claude Code · Scheduled Tasks and Dispatch · GitHub and Netlify · Routines and Pull Requests · Hackathon Step Two

About This Session

Day eleven was a deep dive on the whole Claude ecosystem, taught by Vishwas, YouTuber, content creator, AI generalist, and a co-mentor the cohort had met in the breakout rooms, with Shasti hosting and the hackathon waiting at the end. There was no workbook, for two reasons Shasti had trailed the day before: the hackathon officially opens tonight, so the breakout rooms belong to teams meeting for the first time, and Claude Code needs a paid Claude plan. Vishwas opened by showing the destination, a finance dashboard a previous cohort had named Richie Rich, live on the internet with holdings, liabilities, a FIRE calculator and backups, and promised the room would be in the top one percent of Claude users by the end. His rules were his own: questions to the Q&A; box, the chat reserved for yes-or-no alignment checks, and prompts deliberately conversational, because the skill is not any particular prompt but letting your heart out to a friend who needs to know exactly what you have in mind.

HOW TO USE THESE NOTES

Two additional workbooks sit under day eleven resources on the platform: the FinWin finance tracker built with Claude Code and deployed on Netlify, with every prompt and step, which requires a paid Claude plan, and a Claude Routine workbook that delivers daily updates to your email. Vishwas also promised a document of every prompt he used, including the Claude Design prompt.

Hackathon step two is live: the hackathon portal linked in chat and under the platform's Hackathon section has three tabs, Find a Group, Hackathon Info, and Submit Project. The timer runs from 11.59 pm IST tonight to Sunday the 27th at 10 am IST, and Monday's bonus session connects Claude Code to the Terminal.

Four Friends, and a Plate of Pasta

Ten minutes of theory came with a demand for full attention, because the mental model would carry the day. Claude has four tools and they are four friends, each capable of something different. Claude Chat is the smartest person among eight billion, living on the far side of the world: it knows everything and can only talk, never touching a file, the long-distance advisor. Claude Design is the younger friend who found a passion for visuals and learned nothing else: narrow and deep, it does the work itself, on its own canvas, and hands you the painting. Claude Code is the same kind of specialist in coding, narrow and deep, and it touches the folder you hand over, reading, writing, adding, and deleting files on your own machine. Claude Cowork is your best friend: it knows everything, does the work from start to finish, reaches your tools, files, and apps, and delegates to Code or Design when a task needs the specialist. The pasta analogy sealed it. The chef friend on a call telling you to boil, mix, and correct your mistakes is Chat. A hired chef who cooks in your kitchen with skills you do not possess is Design or Code. And the friend who is chef, negotiator, and mother at once, who knows you had a bad day and wants pasta, buys the missing utensils, orders the missing ingredients on a delivery app, picks the movie while the sauce reduces, and can still summon the master chef if needed, is Cowork. The wide-versus-deep drawing made the same point: the person who knows a bit of everything and the person who knows one thing profoundly are both valuable at the party.

Requirements From Claude Chat

The build began not in Design but in Chat, on the principle that an AI writing for an AI does better than a human trying to articulate everything, and that humans rarely know the most important questions to ask. The chat named the product, FinWin winning over Money Heist and What's Up Buffett, and a Whisper Flow prompt described it in its most nascent form: a personal finance and investment dashboard that tracks assets and liabilities, never connected to real bank accounts, every entry typed by hand; ask me the five most important questions before I design it, then write a complete Claude Design prompt for a responsive prototype covering dashboard, holdings, liabilities, FIRE calculators, and backup settings, with dummy data and every calculation assumption visible. The five questions came back and were answered one by one: currency, the ten most-traded currencies in a settings dropdown with USD default; assets, stocks, mutual funds, FDs, NPS, gold, real estate, crypto, cash, all chosen from dropdowns so nobody types gold every time; storage, data stays in the browser on one device, never on a cloud, downloadable as both JSON and CSV; FIRE, the regular calculator with current age, target retirement age, and monthly expenses, explained as twenty-five times annual expenses working for you at a four percent withdrawal; and hosting, built in Claude Design, coded in Claude Code, pushed through GitHub and Netlify to a public link where every user's data stays in their own browser. A design brief was added on a whim, premium monochrome magenta with subtle cyan accents and a light-dark toggle, precisely to see whether the model could pull off a wild combination, and Opus 5.5, days old and the fastest Opus he had used, returned a hundred and seventy-three lines of prompt in under two minutes, with one flag of its own, INR is not a top-ten currency, added as an eleventh. The lesson underneath was hallucination: it lives in the gaps you leave, in half-answered questions and unstated assumptions, so the human holds the command of creativity and defines everything down to the most basic detail.

Cowork and the Messy Folder

While Design rendered, Cowork got its demonstration on a folder that deserved it, KC Accounts Dump, sixty-odd files across CSVs, PDFs, Word documents, videos, HEIF images, payment screenshots, WhatsApp exports, and subfolders. In the Claude desktop app, never the web, a new chat toggled from Chat to Cowork, the folder was dragged in and allowed, and the prompt was written the way you would brief an assistant: this folder is extremely messy, has invoices and supply orders I need to worry about, go through every file, understand its content, sort into subfolders named for what they hold, rename by date and client for invoices, and do not delete anything. The three approval levels got their explanation: manual approval stops every thirty seconds to ask, which defeats delegation; automatic approval, the recommended setting, runs on its own and pauses only when something looks unsafe; and skip all approvals is for six-hour tasks on machines with nothing to lose, not recommended for at least six months of experience. Commands ran on his own machine, the progress panel ticked through reading and classifying seventy-eight items, and five minutes later a new folder, Kettle and Crumb Accounts, held invoices sorted into rent and utilities, services and repairs, software subscriptions and supplier invoices, each renamed by date and client, plus credit notes, quotes, and supply orders, an index of what went where, and a README for anyone new to the organisation, with the original folder untouched. Connectors extend the same behaviour to Gmail and Google Drive, a Drive link pasted in place of a folder, and the closing demonstration of the day ran the cleaned invoices through Cowork again for charts, returning in seventy-five seconds August's total bills, what was paid from the bank, what was still owed including September rent, the bank balance through the month, a where-the-money-went pie, and a vendor ranking, all from files on the computer without dragging one into a browser.

A Prototype in Claude Design

The Design interface was toured before the result: a chatbot on the left that accepts references, skills, Figma files, a GitHub codebase, or another Design project, a model selector, and the canvas, presented in a new tab for full scale. FinWin had arrived with demo data, a light-dark toggle, total assets, liabilities, a debt-to-asset ratio, FIRE progress, allocation as pie or table, twelve months of net worth, top holdings, and, unasked, a count-towards-FIRE checkbox on every holding, which Vishwas used to explain that an empty plot of land does not fund retirement until it is rented, proof that Claude understood finance as well as design. Three ways to change it were shown. A typed request, add cyan accents, gradients, glows, and light leaks, updated the design in place. A comment, drawn directly on the net-worth chart with a note asking for hover values, month names, and a highlighted point, arrived in the chat as a marked-up prompt and produced an interactive chart. And Edit, plain manual editing of text, with the caveat that code-driven elements like calculated totals refuse manual edits and need a comment, which is exactly what markup means. Sharing offers PDF, PowerPoint, PNG, MP4, Gamma, Google Slides, a code zip, and Publish as Artifact, a public link that lives inside Claude with a Claude Design badge, no custom domain, and no data collection, fine for viewing and self-use but not a functional website. The route to a real website is Send to Claude Code, which generates a prompt to copy into a local agent.

Building in Claude Code

Claude Code lives in the same desktop app behind a Chat-or-Code toggle in the sidebar, and, like Cowork, needs a folder, because a website is never one file: local mode edits files on your machine, cloud mode works in a remote environment such as GitHub. An empty folder was created and trusted as the workspace, the Design prompt pasted, and the instruction added in plain words: implement this design as a deployable finance tracker in the attached folder, preserve every agreed feature, keep entries in browser local storage, test that calculations are correct, that editing works, that entries survive a refresh, that backup and restore work, and do not push or deploy until I say so, show me your plan and ask what you need. The permission modes were explained, manual, accept edits, plan, bypass, and auto, with plan chosen first because a plan costs five to ten percent more tokens and saves double the tokens of a build that misses, then auto once approved. Reality intervened usefully: the desktop app and the web Design were on different Google accounts, so the design connector could not fetch the project, and the fallback was shown, download the Design project as a zip, drop it in the workspace, and tell Code the files are in the folder, with plain HTML, CSS, and JavaScript chosen for a static site. The plan came back with verifications already run in Code's built-in browser pane, demo totals, returns, formatting, backup, mobile layout, a confirm-before-delete guard, and was accepted; nineteen minutes and thirty thousand tokens later, over the break, the FinWin app folder held index.html, styles, scripts, and tests, checked on localhost before anything went live.

Scheduled Tasks, Dispatch, and Routines

While Code built, Cowork's Scheduled tab got its own build. A schedule is an alarm clock, a specific day, date, or time, with templates for daily briefings, weekly reviews, content ideas, and meeting prep, or a new task created with Claude through a short interview: what should each run do, when, and where should results go. The task was a daily 8 am search for the three most important tech and AI stories of the previous day, delivered to a Google Doc in a Drive folder rather than a folder on the computer, because a laptop that is off cannot receive a file, whereas research that lives on Claude's servers runs whether the machine is on or not; tasks that touch local folders are the exception and need the machine awake. Run Now proved it, a doc appeared in Drive with the day's three stories, and the 8 am schedule was left armed. Dispatch, still in beta, is the phone-side companion: a four-hour Code task keeps running on the laptop while approvals and small edits are handled from a phone anywhere. Commands over WhatsApp or Telegram, asked about in chat, are not native and need an automation platform in between. Routines, in Claude Code under More, are the second mechanism and differ in one word: a routine runs on a trigger, not a clock, a schedule, a GitHub event, or an API call, explained through UPI, where scanning a code and entering a PIN reaches the bank through a long alphanumeric key that releases the payment.

GitHub, Netlify, and the Pull-Request Reviewer

Deployment was walked the long way on purpose, since a connected Claude Code could have done it in one prompt. On GitHub, a new private repository named FinWin was created, the codebase's twelve files dragged in and committed, with the distinction drawn carefully: private is about the codebase, public is about the website, you use Amazon and Netflix every day without seeing their code. Netlify, described as a gift, hosts for free on a plan with four hundred credits, offers a Claude agent of its own for tiny sites, and otherwise imports a repository from GitHub, reviews configuration, and deploys, an account-mismatch hiccup and a page-not-found error included. The error became the teaching moment: rather than stopping, ask the AI right back what broke, and Claude Code diagnosed that the files sat in a subfolder instead of the repository root, fixed by re-uploading at the root, after which the site was live for the whole cohort, every visitor's data in their own browser. Why GitHub rather than uploading the folder to Netlify directly: changes on GitHub reflect on the site automatically, and teammates can work on the same codebase. Then the routine that made teamwork safe. A new routine named FinWin Reviewer was created on the cloud, pointed at the repository, its unneeded connectors switched off to save credits, its trigger set to pull request opened, and its instructions written conversationally: review only what this pull request changes, check whether it breaks the net-worth maths, the FIRE calculators, the app's interconnections, or any file the app depends on, or exposes personal data or keys, leave one comment, looks good with a reason or the exact fix, and never change the code yourself. Branches were explained as a parallel duplicate for changes big enough to regret, a branch was created, the page title changed to FinWin Pro on that branch, and a pull request opened, at which point the routine woke on its own and commented within a second that the change was a purely cosmetic single line touching no maths, wiring, or secrets, while Netlify noted nothing would deploy until the lead merged. The merge was clicked, the code changed on main, and the recap ran the whole chain back: Chat for questions, Design for the prototype, comment and edit for changes, Code for the build, GitHub for the codebase, Netlify for the web, schedules for clocks, routines for triggers, and Cowork for everything on your machine.

THE MENTAL MODEL

Chat knows everything and only talks; Design and Code are specialists who do the work in their own domain; Cowork is the best friend who does everything and delegates. Let Chat ask the questions before Design draws, define every detail so nothing is assumed, give Code an empty folder and a plan before a build, put the codebase on GitHub and the site on Netlify, and automate with schedules for clocks and routines for events.

Hackathon Step Two: Rules, Ideas, Submission, and Judging

After the break Uthappa took the hackathon from step one to step two in a briefing he asked the room to rewatch if anything was missed. Is it mandatory? No. Should you attend? A hundred percent yes, because eleven days of accumulated skills need a testing ground, and forty-eight hours in a team of strangers across time zones and backgrounds is where an idea comes to life and ships, the single experience learners from past cohorts most want to repeat; the confident should make space for the less confident, and anyone hesitant should at least watch their team build. It costs nothing, since Google AI Studio, Lovable's free credits across ten accounts, Bolt's ten thousand credits, and GitHub cover it. The rules: a group activity with one product submitted per team, individual entries not evaluated unless you are the only active member, a minimum viable product that works with bare-minimum features, any tools and any stack, and an idea that must come from the provided list, personal health, personal finance, a collaborative note-taker, a task-management app, AI interior makeover, a resume-to-interview coach, a pet-care companion, or a campaign planner, with a three-phase product-building playbook and a worked travel-itinerary example attached for those without a product background. The hackathon portal holds it all under three tabs. Find a Group works as yesterday, registered email in, group number and WhatsApp link out, raise a request if not found. Hackathon Info holds the vibe-coding playbook, the idea list, and the pitch-deck template, which teams copy and fill with team details and roles, the problem statement, a screenshot of the hero page or main feature, and a five-minute video on Loom or any recorder. Submit Project carries the live timer and the form: group number, one team leader who orchestrates like a master agent, every member's full name for the certificates, product name and description with nothing left as NA, the product URL with open access or a specific demo login, the video link, and the deck link. The window opens at 11.59 pm IST tonight, exactly one hour after the briefing, and because Friday and Saturday carry full sessions, the forty-eight hours were extended by ten to Sunday the 27th at 10 am IST, with the timer on the portal as the single source of truth across time zones. Judging weighs four things equally, understanding of the problem, functionality, the creative angle, and presentation, interface aesthetics and storytelling in the deck and video, and the top three win a certificate, ten minutes each to demo on graduation day at 7.30 pm IST on Sunday, and a place on the alumni showcase page Outskill is building into its website.

The Next Three Days, and the Team Rooms

The closing map covered the busiest stretch of the fortnight. Friday brings a dedicated monetisation session and, after it, an introduction to the community of ten thousand-plus AI generalists and builders the cohort now joins, the platforms for reaching it, and a networking product built to match learners by interest and objective. Saturday teaches building your own AI Jarvis with OpenClaw and Hermes, then drops the link to book a one-to-one alumni onboarding call with the practitioner and programme teams over the following week, mirroring the call that began the journey, alongside the next steps and the roadmap Vaibhav had promised. Sunday is graduation: Dileep on picking a problem statement from your own daily life, the hackathon winners' demos, learner stories, and the certificate. Tonight's rooms were self-selected, join the breakout room whose number matches your WhatsApp group, cameras on, introduce yourselves, and pick one problem statement, forty-five minutes only, then carry the conversation to the WhatsApp group or your own Meet link, while anyone still without a group dropped an email in the Q&A; box. The rooms were pulled back at midnight IST with Uthappa's reminder to plan fast and start building.

Key Takeaways

Stripped to its essentials, the session made the following points.

- **Four friends, four jobs.** Chat advises from afar, Design and Code are specialists who do the work in their own domain, and Cowork is the best friend with all the context who does everything and delegates to the specialists.
- **Ask before you design.** Have Chat interview you on the five decisions that matter, answer them fully, and let it write the Design prompt, because hallucination lives in every question you leave half-answered.
- **Cowork works on your machine.** Attach a folder, brief it like an assistant, choose automatic approval, and it sorts sixty files with an index and a README, or charts a month of invoices, in minutes, with connectors doing the same for Drive and Gmail.
- **Design changes three ways.** Prompt for the sweeping change, comment on a drawn area for the specific one, edit text by hand, and remember that calculated elements only respond to markup.
- **Code needs a folder and a plan.** Trust an empty workspace, hand over the design or its zip, ask for a plan before the build, run auto once approved, and check localhost and the built-in browser before anything goes live.
- **Schedules are clocks, routines are triggers.** A daily brief to Drive runs even with the laptop off, Dispatch approves from your phone, and a routine on pull request opened reviews every teammate's change.
- **GitHub holds the code, Netlify serves the site.** Private codebase, public website, files at the repository root, and when a deploy breaks, ask the AI what broke.
- **The hackathon is open.** Teams of strangers, one listed idea, an MVP in any stack, a deck and a five-minute video submitted through the portal by Sunday 10 am IST, judged equally on problem, function, creativity, and presentation.

End of Day Eleven notes. Next: monetisation, and the alumni community — Day Twelve.